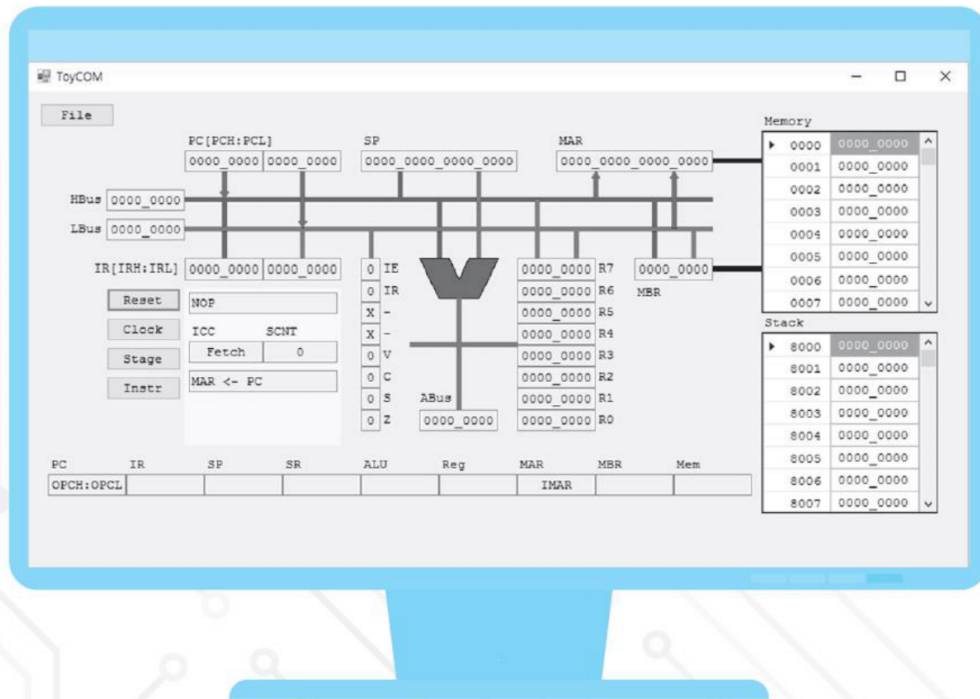


한눈에 보이는 컴퓨터 구조

개정 3 판



시험범위 X

07. 중앙처리장치 설계

7.1 프로그래머 모델

7.2 명령어 형식

7.3 명령어 집합

7.4 명령어 사이클

7.5 마이크로오퍼레이션

7.6 제어신호

7.7 요약

7.1 프로그래머 모델

ToyCOM 사양

- 8비트 프로세서
 - 8 개의 범용 레지스터 (R0 ~ R7)
 - 8비트 상태 레지스터
- 기억장치 용량 64 KB
 - 주소 버스 16 비트: 16비트 MAR
 - 데이터 버스 8 비트: 8비트 MBR
- 16비트 명령어

프로그래머 모델

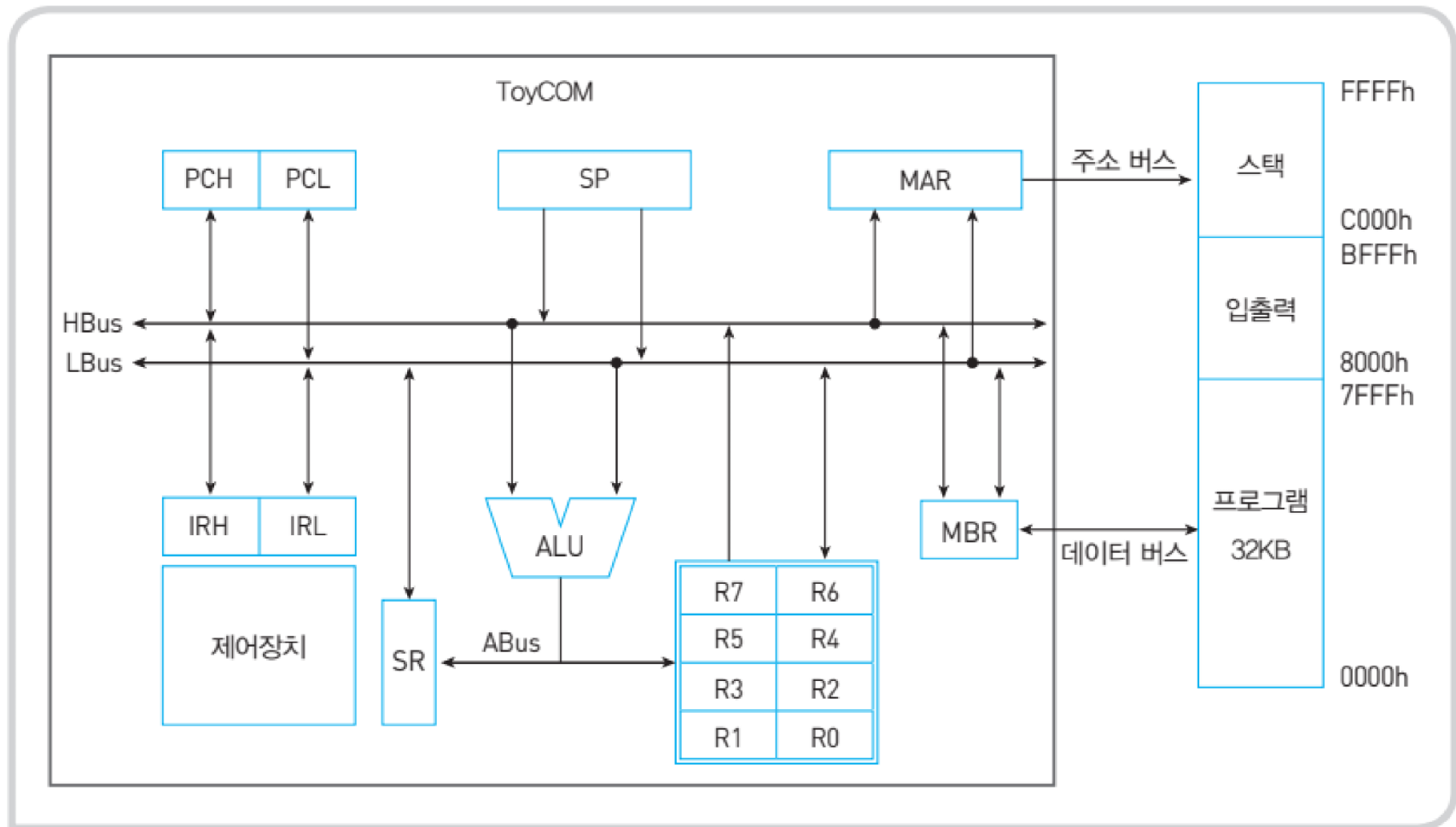


그림 7-1 ToyCOM 프로그래머 모델

레지스터

16비트 프로그램 카운터(PC)

- 8비트씩 액세스할 수 있도록, PCH와 PCL로 분리

16비트 스택 포인터(SP)

16비트 명령어 레지스터(IR)

- 8비트씩 액세스할 수 있도록, IRH와 IRL로 분리

8비트 상태 레지스터

8개의 범용 레지스터(R7:R0)

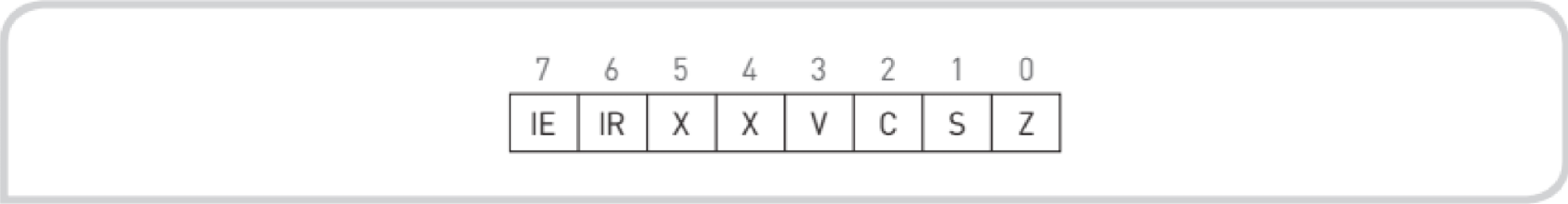
- 정렬된 레지스터 쌍으로 주소 지정
- 예) R1:R0, R3:R2, R5:R4, R7:R6

16비트 기억장치 주소 레지스터(MAR)

8비트 기억장치 버퍼 레지스터(MBR)

- HBus 또는 LBus로 연결

상태 레지스터



7	6	5	4	3	2	1	0
IE	IR	X	X	V	C	S	Z

그림 7-2 상태 레지스터

플래그 종류

- Zero flag
- Sign flag
- Carry flag
- oVerflow flag
- IR (Interrupt Request flag)
- IE (Interrupt Enable flag)

표현

- (상태 레지스터).(플래그 이름)
- 예) SR.C, SR.IR

7.2 명령어 형식

명령어 형식

- 기억장치 단어의 2배(16비트)인 고정 길이 명령어
- 데이터 처리 명령어는 레지스터만으로 동작
- 적재(load)와 저장(store)만으로 기억장치 액세스
- 기억장치를 액세스할 때는 레지스터 간접 주소지정방식 사용
- 프로그램 제어 명령어는 상대 주소지정방식으로 분기 목적지 표현

명령어 형식의 필드

- 동작코드(opcode): 명령어 또는 명령어 그룹을 구분하는 코드
- 즉치(immediate): 즉치 주소지정방식에서 명령어에 포함되는 상수
- 부동작코드(sop1, sop2): 같은 그룹의 명령어를 세분화하는 코드
- 목적지 레지스터(Rd): 명령어의 목적지 레지스터 번호
- 소스 레지스터(Rs1, Rs2): 명령어의 소스 레지스터 번호
- 변위(offset): 상대 주소지정방식의 변위

명령어 형식



그림 7-3 ToyCOM 명령어 형식

opcode: 5bit
register: 3bit

형식 1: 즉시 데이터 전달 명령어

형식 2: 데이터 전달 명령어, 단항 데이터 처리 명령어

형식 3: 이항 데이터 처리 명령어

형식 4: 프로그램 제어 명령어

명령어 종류

표 7-1 ToyCOM의 명령어 종류

명령어 종류	형식	주소지정방식	명령어 니모닉 코드	수
데이터 전달	형식 1	즉치	LDI	1
	형식 2	레지스터 간접	LD, ST	2
		레지스터	MV	1
		스택	PUSH, POP	2
데이터 처리	형식 2	레지스터	INC, DEC, NEG, NOT, SHL, SHR, ASL, ASR	8
	형식 3	레지스터	ADD, ADC, SUB, SBC, AND, OR, XOR	7
비교/ 플래그 설정	형식 3	레지스터	CMP	1
		없음	CLC, STC, CLI, STI	4
프로그램 제어	형식 4	상대	BR, BRNZ, BRZ, BRNS, BRS, BRNC, BRC, BRNV, BRV, BBE, BAE, BL, BLE, BG, BGE, CALL	16
		없음	RET, RETI	2
시스템 관리	형식 4	없음	NOP, HALT	2
합계				46

주소지정방식

즉치 주소지정방식

- 즉치 데이터 적재 명령어(LDI)가 사용한다.

레지스터 간접 주소지정방식

- 기억장치에 유효 데이터가 있는 적재와 저장 명령어가 사용한다.
- 레지스터 쌍에 기억장치 주소를 저장한다.

레지스터 주소지정방식

- 레지스터 간 데이터 이동, 단항과 이항 연산 명령어, 비교 명령어가 사용한다.

상대 주소지정방식

- 무조건분기, 조건분기, 서브루틴 호출 명령어가 사용한다.
- 프로그램 카운터 값과 분기 목적지 주소 간의 거리를 변위(offset) 필드로 표현한다.
- 변위의 범위는 $\pm 1\text{KB}$

7.3 명령어 집합

ToyCOM 명령어 소개

내용

- 7.3.1 데이터 전달 명령어
- 7.3.2 데이터 처리 명령어
- 7.3.3 비교 및 프래그 설정 명령어
- 7.3.4 분기 명령어
- 7.3.5 서브루틴 호출 및 인터럽트 명령어
- 7.3.6 시스템 관리 명령어

7.3.1 데이터 전달 명령어

표 7-3 데이터 전달 명령어

명령어	비트 패턴					동작
	15 - 11	10 - 8	7 - 5	4 - 2	1 - 0	
LDI Rd, #value	00001	Rd	#value			$Rd \leftarrow \#value$
LD Rd, (Rs+1:Rs)	00010	Rd	Rs	000	00	$Rd \leftarrow \text{Mem}(Rs+1:Rs)$
ST (Rd+1:Rd), Rs	00010	Rd	Rs	000	01	$\text{Mem}(Rd+1:Rd) \leftarrow Rs$
MV Rd, Rs	00010	Rd	Rs	001	00	$Rd \leftarrow Rs$
PUSH Rs	00011	X	Rs	000	00	$SP \leftarrow SP - 1 / \text{Mem}(SP) \leftarrow Rs$
POP Rd	00011	Rd	X	000	01	$Rd \leftarrow \text{Mem}(SP) / SP \leftarrow SP + 1$

데이터 전달 명령어

[예제 7-1] 레지스터와 기억장치 상태가 그림과 같다.

- 1) 1100h 번지 명령어 실행 후 변하는 레지스터의 값은?
- 2) 1102h 번지 명령어 실행 후 변하는 레지스터의 값은?
- 3) 1104h 번지 명령어 실행 후 변하는 레지스터의 값은?

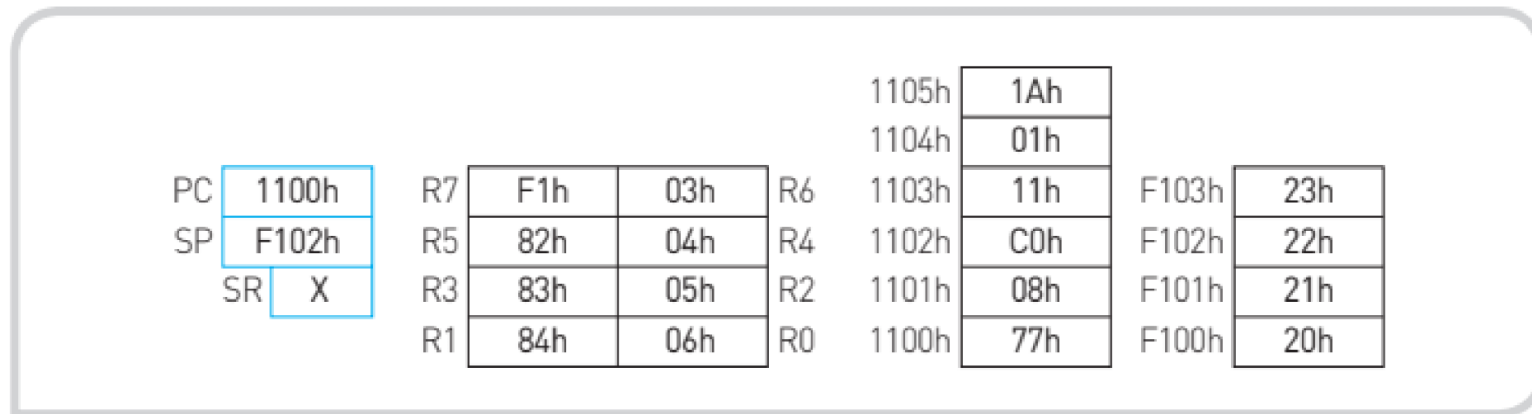


그림 7-4 예제 7-1의 ToyCOM 상태

7.3.2 데이터 처리 명령어

표 7-4 데이터 처리 명령어

명령어	비트 패턴					동작
	15 - 11	10 - 8	7 - 5	4 - 2	1 - 0	
INC Rd	00100	Rd	X	000	00	$Rd \leftarrow Rd + 1$
DEC Rd	00100	Rd	X	000	01	$Rd \leftarrow Rd - 1$
NEG Rd	00100	Rd	X	000	10	$Rd \leftarrow -Rd$
NOT Rd	00100	Rd	X	000	11	$Rd \leftarrow Rd'$
SHL Rd	00100	Rd	X	001	00	$Rd \leftarrow \text{logical shift left } Rd$
SHR Rd	00100	Rd	X	001	01	$Rd \leftarrow \text{logical shift right } Rd$
ASL Rd	00100	Rd	X	001	10	$Rd \leftarrow \text{arithmetic shift left } Rd$
ASR Rd	00100	Rd	X	001	11	$Rd \leftarrow \text{arithmetic shift right } Rd$
ADD Rd, Rs1, Rs2	00101	Rd	Rs1	Rs2	00	$Rd \leftarrow Rs1 + Rs2$
ADC Rd, Rs1, Rs2	00101	Rd	Rs1	Rs2	01	$Rd \leftarrow Rs1 + Rs2 + SR.C$
SUB Rd, Rs1, Rs2	00101	Rd	Rs1	Rs2	10	$Rd \leftarrow Rs1 - Rs2$
SBC Rd, Rs1, Rs2	00101	Rd	Rs1	Rs2	11	$Rd \leftarrow Rs1 - Rs2 - SR.C$
AND Rd, Rs1, Rs2	00110	Rd	Rs1	Rs2	00	$Rd \leftarrow Rs1 \& Rs2$
OR Rd, Rs1, Rs2	00110	Rd	Rs1	Rs2	01	$Rd \leftarrow Rs1 Rs2$
XOR Rd, Rs1, Rs2	00110	Rd	Rs1	Rs2	10	$Rd \leftarrow Rs1 \wedge Rs2$

데이터 처리 명령어

[예제 7-2] 명령어에 대한 명령어 코드를 16진수로 적어라.

- 1) DEC R5
- 2) SHL R2
- 3) ADD R0, R2, R3

[예제 7-3] 16 비트 덧셈

기억장치 [4201h:4200h]와 [4203:4202h]의 16비트 데이터를 더하여
기억장치 [4205:4204h]에 저장하는 어셈블리 프로그램을 작성하라.

LDI	R7, #42h	// R7:R6 = 4200h
LDI	R6, #00h	
LD	R0, (R7:R6)	// R0 = Mem(4200h)
INC	R6	// R7:R6 = 4201h
LD	R1, (R7:R6)	// R1 = Mem(4201h)
INC	R6	// R7:R6 = 4202h
LD	R2, (R7:R6)	// R2 = Mem(4202h)
INC	R6	// R7:R6 = 4203h
LD	R3, (R7:R6)	// R3 = Mem(4203h)
ADD	R4, R0, R2	// R4 = R0 + R2
ADC	R5, R1, R3	// R5 = R1 + R3 + C
INC	R6	// R7:R6 = 4204h
ST	(R7:R6), R4	// Mem(4204h) = R4
INC	R6	// R7:R6 = 4205h
ST	(R7:R6), R5	// Mem(4205h) = R5

7.3.3 비교 및 플래그 설정 명령어

표 7-5 비교 및 플래그 설정 명령어

명령어	비트 패턴					동작
	15 - 11	10 - 8	7 - 5	4 - 2	1-0	
CMP Rs1, Rs2	00111	X	Rs1	Rs2	X	Rs1 - Rs2
CLC	01000	X	X	X	00	SR.C ← 0
STC	01000	X	X	X	01	SR.C ← 1
CLI	01000	X	X	X	10	SR.IE ← 0
STI	01000	X	X	X	11	SR.IE ← 1

7.3.4 분기 명령어

상대 주소지정방식

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode					offset										

- 분기 목적지 주소 = PC + offset (11 비트)
- 분기 목적지: $PC - 1024 \leq \text{분기 목적지 주소} \leq PC + 1023$

분기 명령어

- 무조건 분기 명령어
- SR 조사 조건 분기 명령어
- 부호 없는 수 조건 분기 명령어
- 정수 조건 분기 명령어

분기 명령어

표 7-6 상대 주소지정방식에 의한 분기 명령어

명령어	비트 패턴		동작
	15 - 11	10 - 0	
BR target	10000	offset	$PC \leftarrow PC + \text{offset}$
BRNZ target	10001	offset	$Z=0: PC \leftarrow PC + \text{offset}$
BRZ target	10010	offset	$Z=1: PC \leftarrow PC + \text{offset}$
BRNS target	10011	offset	$S=0: PC \leftarrow PC + \text{offset}$
BRS target	10100	offset	$S=1: PC \leftarrow PC + \text{offset}$
BRNC target	10101	offset	$C=0: PC \leftarrow PC + \text{offset}$
BRC target	10110	offset	$C=1: PC \leftarrow PC + \text{offset}$
BRNV target	10111	offset	$V=0: PC \leftarrow PC + \text{offset}$
BRV target	11000	offset	$V=1: PC \leftarrow PC + \text{offset}$
BRE target	10010	offset	$Z=1: PC \leftarrow PC + \text{offset}$
BRA target	11001	offset	$C \mid Z': PC \leftarrow PC + \text{offset}$
BRAE target	10110	offset	$C=1: PC \leftarrow PC + \text{offset}$
BRBE target	11010	offset	$C' \mid Z: PC \leftarrow PC + \text{offset}$
BRB target	10101	offset	$C=0: PC \leftarrow PC + \text{offset}$
BRGT target	11011	offset	$(Z \mid (S \oplus V))': PC \leftarrow PC + \text{offset}$
BRGE target	11100	offset	$(S \oplus V)': PC \leftarrow PC + \text{offset}$
BRLE target	11101	offset	$Z \mid (S \oplus V): PC \leftarrow PC + \text{offset}$
BRLT target	11110	offset	$S \oplus V: PC \leftarrow PC + \text{offset}$

분기 명령어

[예제 7-4]

- 기억장치 [2109:2108]의 명령어가 87E0일 때, 명령어 실행 후 PC 값은?

[풀이]

- 명령어 실행시 PC = 210Ah
- 명령어 해석: 87E0h = 10000_111_1110_0000 = BR target(offset)
- Offset = 111_1110_0000 = -20h
- $PC \leftarrow PC + \text{offset} = 210Ah - 20h = 20EAh$

7.3.5 서브루틴 호출 및 인터럽트 명령어

표 7-7 서브루틴 호출 및 인터럽트 명령어

명령어	비트 패턴				동작
	15 - 11	10 - 8	7 - 2	1 - 0	
CALL target	11111	offset			PUSH PC, $PC \leftarrow PC + \text{offset}$
RET	01001	X	X	00	POP PC
RETI	01001	X	X	01	POP PC, POP SR

서브루틴 호출 예제

[예제 7-5] 명령어 실행 후 ToyCOM 상태는?

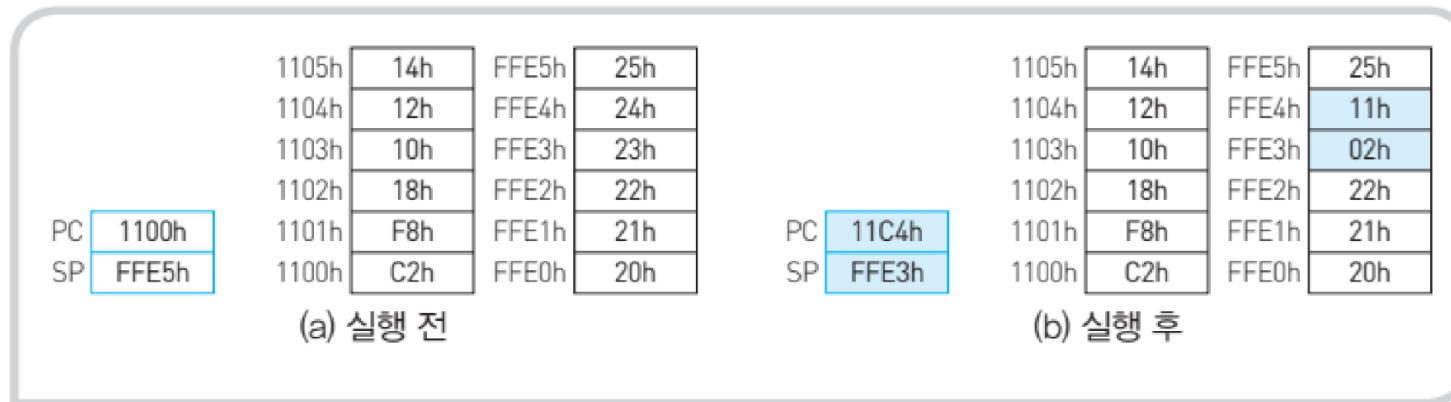


그림 7-5 예제 7-5의 ToyCOM 상태

명령어 인출

- $PC \leftarrow 1102h$
- $IR \leftarrow F8C2h = 11111_000_1100_0010$
 $= CALL\ PC+0C2h$
- $Target = 1102h + 0C2h = 11C4h$

• 명령어 실행

- $SP \leftarrow SP - 1 = FFE4h$ // PUSH PC
- $Mem[SP] \leftarrow PC[15:8] = 11h$
- $SP \leftarrow SP - 1 = FFE3h$
- $Mem[SP] \leftarrow PC[7:0] = 02h$
- $PC = 11C4h$

7.3.6 시스템 관리 명령어

표 7-8 시스템 관리 명령어

명령어	비트 패턴			동작
	15 - 11	10 - 2	1 - 0	
NOP	00000	X	00	No operation
HALT	00000	X	01	System halt

7.4 명령어 사이클

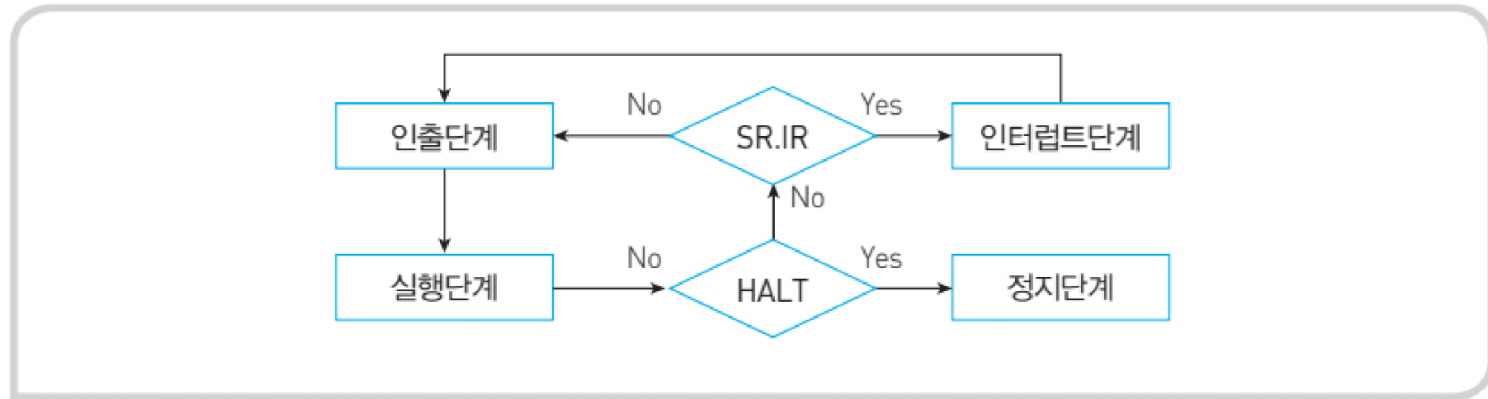


그림 7-6 ToyCOM 명령어 사이클

인출단계	2 바이트 명령어를 IR에 저장한다.
실행단계	명령어를 실행하고, SR.IR에 따라 다음 단계 결정한다.
정지단계	HALT 명령어를 실행하면 시스템을 정지한다.
인터럽트단계	SR.IR=1일 때, 인터럽트 단계로 이동한다.

인터럽트 벡터

인터럽트 종류

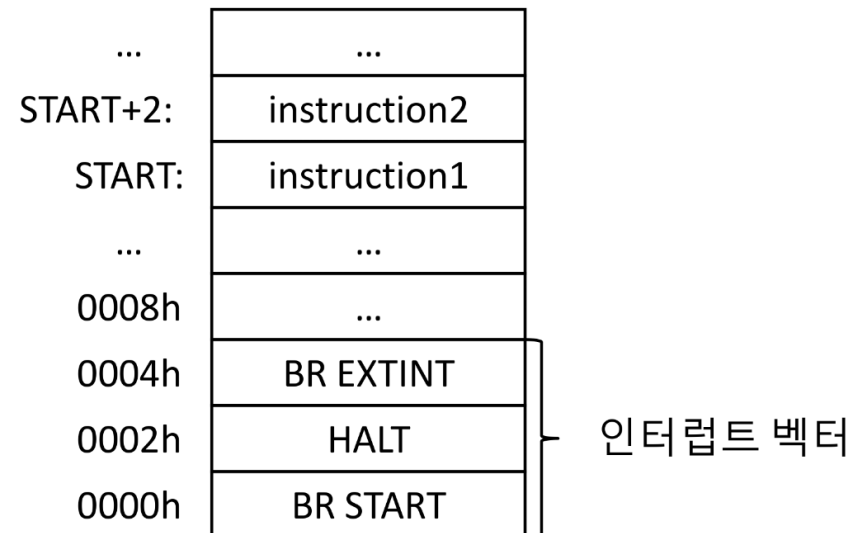
- 0 번: 시스템 리셋
- 1 번: HALT 명령어
- 2 번: 외부 인터럽트

인터럽트 벡터

- ISR 시작 주소의 모임

인터럽트 발생시

- $PC \leftarrow 2 \times \text{인터럽트 번호}$



7.5 마이크로오퍼레이션

마이크로오퍼레이션

- 한 개의 클럭 사이클 구간에 수행하는 동작

각 명령어 사이클 단계에 대한 마이크로오퍼레이션을 정의한다.

내용

- 7.5.1 인출단계
- 7.5.2 실행단계
- 7.5.3 정지단계
- 7.5.4 인터럽트단계

7.5.1 인출 단계

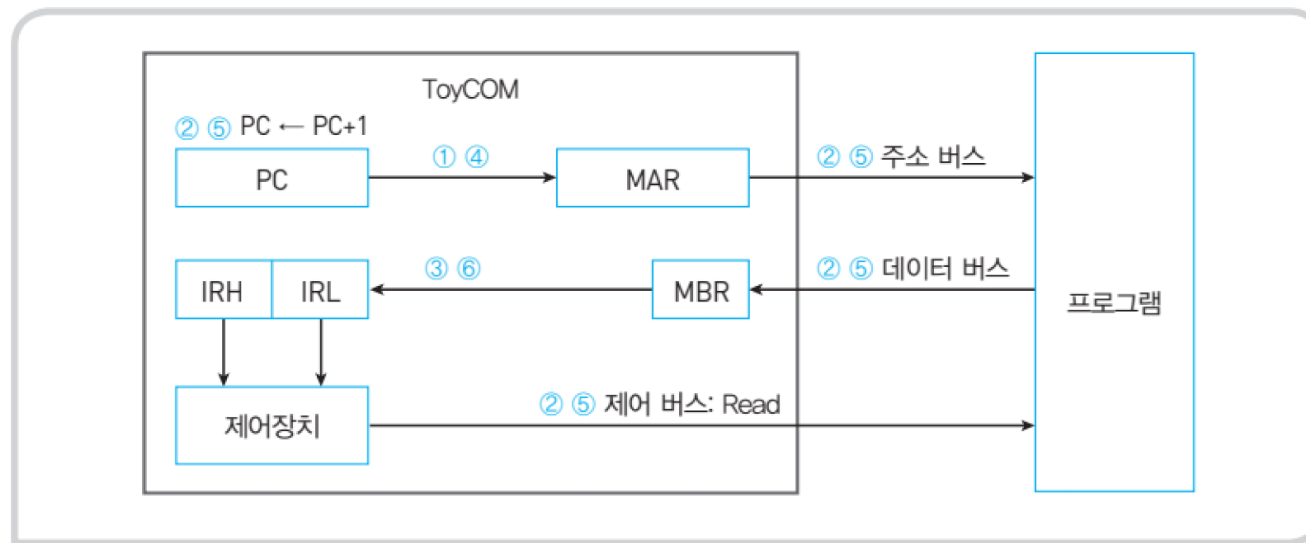


그림 7-8 명령어 인출 마이크로오퍼레이션

표 7-9 인출단계 마이크로오퍼레이션

명령어 사이클	동작	마이크로오퍼레이션
인출단계	$IR \leftarrow Mem[PC]$ $PC \leftarrow PC + 2$	1. $MAR \leftarrow PC$ 2. $MBR \leftarrow Mem(MAR), PC \leftarrow PC + 1$ 3. $IRL \leftarrow MBR$ 4. $MAR \leftarrow PC$ 5. $MBR \leftarrow Mem(MAR), PC \leftarrow PC + 1$ 6. $IRH \leftarrow MBR$

7.5.2 실행 단계

명령어 별로 마이크로오퍼레이션 정의

- 데이터 전달 명령어
- 데이터 처리 명령어
- 비교 및 플래그 설정 명령어
- 분기 명령어
- 서브루틴 호출 및 인터럽트 명령어
- 시스템 관리 명령어

프로그래머 모델 고려

데이터 전달 명령어

표 7-10 데이터 전달 명령어의 마이크로오퍼레이션

명령어	동작	마이크로오퍼레이션
LDI Rd, #value	$Rd \leftarrow \#value$	1. $Rd \leftarrow IRL$
LD Rd, (Rs+1:Rs)	$Rd \leftarrow Mem(Rs+1:Rs)$	1. $MAR \leftarrow Rs+1:Rs$ 2. $MBR \leftarrow Mem(MAR)$ 3. $Rd \leftarrow MBR$
ST (Rd+1:Rd), Rs	$Mem(Rd+1:Rd) \leftarrow Rs$	1. $MAR \leftarrow Rd+1:Rd$ 2. $MBR \leftarrow Rs$ 3. $Mem(MAR) \leftarrow MBR$
MV Rd, Rs	$Rd \leftarrow Rs$	1. $Rd \leftarrow Rs$
PUSH Rs	$SP \leftarrow SP - 1$ $Mem(SP) \leftarrow Rs$	1. $SP \leftarrow SP - 1$ 2. $MAR \leftarrow SP$ 3. $MBR \leftarrow Rs$ 4. $Mem(MAR) \leftarrow MBR$
POP Rd	$Rd \leftarrow Mem(SP)$ $SP \leftarrow SP + 1$	1. $MAR \leftarrow SP$ 2. $MBR \leftarrow Mem(MAR)$ 3. $Rd \leftarrow MBR, SP \leftarrow SP + 1$

데이터 처리 명령어

표 7-11 데이터 처리 명령어의 마이크로오퍼레이션

명령어	동작	마이크로오퍼레이션
INC Rd	$Rd \leftarrow Rd + 1$	$Rd \leftarrow Rd + 1$
DEC Rd	$Rd \leftarrow Rd - 1$	$Rd \leftarrow Rd - 1$
NEG Rd	$Rd \leftarrow -Rd$	$Rd \leftarrow -Rd$
NOT Rd	$Rd \leftarrow Rd'$	$Rd \leftarrow Rd'$
SHL Rd	$Rd \leftarrow \text{logical shift left } Rd$	$Rd \leftarrow \text{SHL } Rd$
SHR Rd	$Rd \leftarrow \text{logical shift right } Rd$	$Rd \leftarrow \text{SHR } Rd$
ASL Rd	$Rd \leftarrow \text{arithmetic shift left } Rd$	$Rd \leftarrow \text{ARL } Rd$
ASR Rd	$Rd \leftarrow \text{arithmetic shift right } Rd$	$Rd \leftarrow \text{ASR } Rd$
ADD Rd, Rs1, Rs2	$Rd \leftarrow Rs1 + Rs2$	$Rd \leftarrow Rs1 + Rs2$
ADC Rd, Rs1, Rs2	$Rd \leftarrow Rs1 + Rs2 + \text{SR.C}$	$Rd \leftarrow Rs1 + Rs2 + \text{SR.C}$
SUB Rd, Rs1, Rs2	$Rd \leftarrow Rs1 - Rs2$	$Rd \leftarrow Rs1 - Rs2$
SBC Rd, Rs1, Rs2	$Rd \leftarrow Rs1 - Rs2 - \text{SR.C}$	$Rd \leftarrow Rs1 - Rs2 - \text{SR.C}$
AND Rd, Rs1, Rs2	$Rd \leftarrow Rs1 \& Rs2$	$Rd \leftarrow Rs1 \& Rs2$
OR Rd, Rs1, Rs2	$Rd \leftarrow Rs1 Rs2$	$Rd \leftarrow Rs1 Rs2$
XOR Rd, Rs1, Rs2	$Rd \leftarrow Rs1 \wedge Rs2$	$Rd \leftarrow Rs1 \wedge Rs2$
CMP Rs1, Rs2	$\text{SR} \leftarrow Rs1 - Rs2$	$\text{SR} \leftarrow Rs1 - Rs2$
CLC	$\text{SR.C} \leftarrow 0$	$\text{SR.C} \leftarrow 0$
STC	$\text{SR.C} \leftarrow 1$	$\text{SR.C} \leftarrow 1$
CLI	$\text{SR.IE} \leftarrow 0$	$\text{SR.IE} \leftarrow 0$
STI	$\text{SR.IE} \leftarrow 1$	$\text{SR.IE} \leftarrow 1$

분기 명령어

표 7-12 분기 명령어의 마이크로오퍼레이션

명령어	동작	마이크로오퍼레이션
BR target	$PC \leftarrow PC + \text{offset}$	$PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRNZ target	$Z=0: PC \leftarrow PC + \text{offset}$	$Z=0: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRZ target	$Z=1: PC \leftarrow PC + \text{offset}$	$Z=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRNS target	$S=0: PC \leftarrow PC + \text{offset}$	$S=0: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRS target	$S=1: PC \leftarrow PC + \text{offset}$	$S=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRNC target	$C=0: PC \leftarrow PC + \text{offset}$	$C=0: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRC target	$C=1: PC \leftarrow PC + \text{offset}$	$C=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRNV target	$V=0: PC \leftarrow PC + \text{offset}$	$V=0: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRV target	$V=1: PC \leftarrow PC + \text{offset}$	$V=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRE target	$Z=1: PC \leftarrow PC + \text{offset}$	$Z=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRA target	$C \mid Z': PC \leftarrow PC + \text{offset}$	$C \mid Z': PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRAE target	$C=1: PC \leftarrow PC + \text{offset}$	$C=1: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRBE target	$C' \mid Z: PC \leftarrow PC + \text{offset}$	$C' \mid Z: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRB target	$C=0: PC \leftarrow PC + \text{offset}$	$C=0: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRGT target	$(Z \mid (S \oplus V))': PC \leftarrow PC + \text{offset}$	$(Z \mid (S \oplus V))': PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRGE target	$(S \oplus V)': PC \leftarrow PC + \text{offset}$	$(S \oplus V)': PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRLE target	$Z \mid (S \oplus V): PC \leftarrow PC + \text{offset}$	$Z \mid (S \oplus V): PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$
BRLT target	$S \oplus V: PC \leftarrow PC + \text{offset}$	$S \oplus V: PC \leftarrow PC + \text{IRH}[10:8]:\text{IRL}$

프로그램 제어 명령어

표 7-13 호출, 복귀 및 시스템 관리 명령어의 마이크로퍼레이션

명령어	동작	마이크로퍼레이션
CALL target	PUSH PC $PC \leftarrow PC + \text{offset}$	1. $SP \leftarrow SP - 1$ // PUSH PCH 2. $MAR \leftarrow SP$ 3. $MBR \leftarrow PCH$ 4. $Mem(MAR) \leftarrow MBR$ 5. $SP \leftarrow SP - 1$ // PUSH PCL 6. $MAR \leftarrow SP$ 7. $MBR \leftarrow PCL$ 8. $Mem(MAR) \leftarrow MBR$ 9. $PC \leftarrow PC + IRH[10:8]:IRL$
RET	POP PC	1. $MAR \leftarrow SP$ // POP PCL 2. $MBR \leftarrow Mem(MAR)$ 3. $PCL \leftarrow MBR, SP \leftarrow SP + 1$ 4. $MAR \leftarrow SP$ // POP PCH 5. $MBR \leftarrow Mem(MAR)$ 6. $PCH \leftarrow MBR, SP \leftarrow SP + 1$
IRET	POP SR POP PC	1. $MAR \leftarrow SP$ // POP SR 2. $MBR \leftarrow Mem(MAR)$ 3. $SR \leftarrow MBR, SP \leftarrow SP + 1$ 4. $MAR \leftarrow SP$ // POP PCL 5. $MBR \leftarrow Mem(MAR)$ 6. $PCL \leftarrow MBR, SP \leftarrow SP + 1$ 7. $MAR \leftarrow SP$ // POP PCH 8. $MBR \leftarrow Mem(MAR)$ 9. $PCH \leftarrow MBR, SP \leftarrow SP + 1$
NOP	없음	없음
HALT	없음	없음

7.5.3 정지 단계

HALT 명령어를 실행할 때

인터럽트 1번 ISR

- Invalid opcode

정지 단계 이후 다음 단계가 없다.

7.5.4 인터럽트 단계

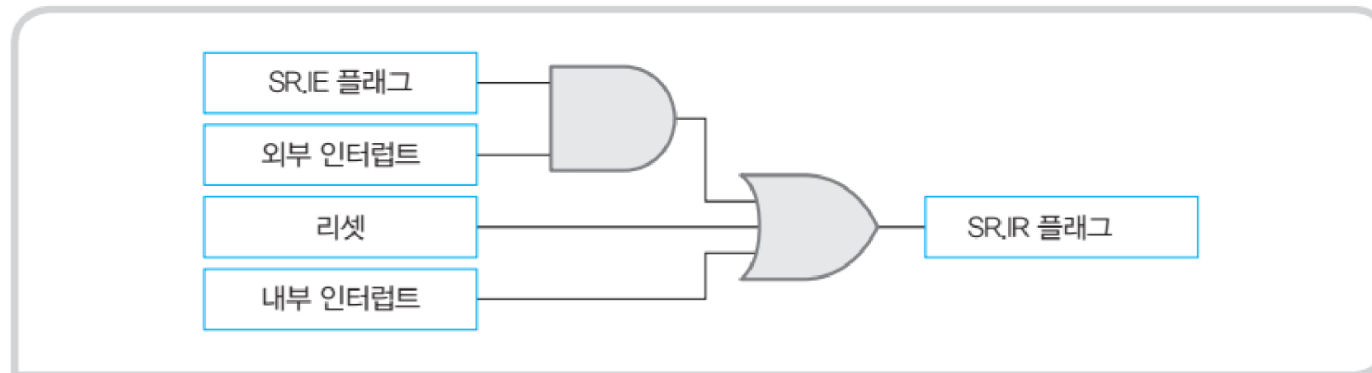


그림 7-9 인터럽트 요청 신호

표 7-14 인터럽트단계의 마이크로오퍼레이션

명령어 사이클	동작	마이크로오퍼레이션
인터럽트단계	PUSH PC PUSH SR $PC \leftarrow 2 \times n$	1. $SP \leftarrow SP - 1$ // PUSH PCH 2. $MAR \leftarrow SP$ 3. $MBR \leftarrow PCH$ 4. $Mem(MAR) \leftarrow MBR$ 5. $SP \leftarrow SP - 1$ // PUSH PCL 6. $MAR \leftarrow SP$ 7. $MBR \leftarrow PCL$ 8. $Mem(MAR) \leftarrow MBR$ 9. $SP \leftarrow SP - 1$ // PUSH SR 10. $MAR \leftarrow SP$ 11. $MBR \leftarrow SR$ 12. $Mem(MAR) \leftarrow MBR, PC \leftarrow 2 \times n$

7.6 제어신호

ToyCOM 프로그래머 모델에 제어신호 추가

- (7.5절) 마이크로오퍼레이션을 실행할 수 있도록
- (7.1절) 프로그래머 모델에 제어신호를 할당한다.

제어신호 추출 과정

- 7.6.1 마이크로오퍼레이션 선택
- 데이터 전달 경로 설계 – 프로그래머 모델
- 7.6.2 제어신호 할당
- 7.6.3 검증

7.6.1 마이크로오퍼레이션 선택

마이크로오퍼레이션 선택

- 7.5절에서 구한 마이크로오퍼레이션은 중복되어 있다.
- 중복되지 않도록 마이크로오퍼레이션을 선택한다.

표 7-15 마이크로오퍼레이션 선택 결과

동작 대상	마이크로오퍼레이션 목록			수
PC	$PCH \leftarrow MBR$ $PC \leftarrow PC + 1$	$PCL \leftarrow MBR$ $PC \leftarrow 2 \times n$	$PC \leftarrow PC + IRH[10:8]:IRL$	5
IR	$IRH \leftarrow MBR$	$IRL \leftarrow MBR$		2
SP	$SP \leftarrow SP - 1$	$SP \leftarrow SP + 1$		2
SR	$SR.C \leftarrow 0$ $SR.IE \leftarrow 0$	$SR.C \leftarrow 1$ $SR.IE \leftarrow 1$	$SR \leftarrow MBR$	5
Rx	$Rx \leftarrow IRL$ $Rx \leftarrow Rx + 1$ $Rx \leftarrow Rx'$ $Rx \leftarrow ARL Rx$ $Rx \leftarrow Rx1 + Rx2 + SR.C$ $Rx \leftarrow Rx1 \& Rx2$	$Rx \leftarrow MBR$ $Rx \leftarrow Rx - 1$ $Rx \leftarrow SHL Rx$ $Rx \leftarrow ASR Rx$ $Rx \leftarrow Rx1 - Rx2$ $Rx \leftarrow Rx1 Rx2$	$Rx \leftarrow Rx$ $Rx \leftarrow -Rx$ $Rx \leftarrow SHR Rx$ $Rx \leftarrow Rx1 + Rx2$ $Rx \leftarrow Rx1 - Rx2 - SR.C$ $Rx \leftarrow Rx1 \wedge Rx2$	18
MAR	$MAR \leftarrow PC$	$MAR \leftarrow SP$	$MAR \leftarrow Rx+1:Rx$	3
MBR	$MBR \leftarrow Rx$ $MBR \leftarrow SR$	$MBR \leftarrow PCL$	$MBR \leftarrow PCH$	4
기억장치	$MBR \leftarrow Mem[MAR]$	$Mem[MAR] \leftarrow MBR$		2
합계				41

데이터 전달 경로 설계

데이터 전달 경로

- 프로그래머 모델은 선택한 마이크로오퍼레이션을 실행할 수 있는 경로를 포함해야 한다. 마이크로오퍼레이션을 실행할 수 있는지 점검한다.

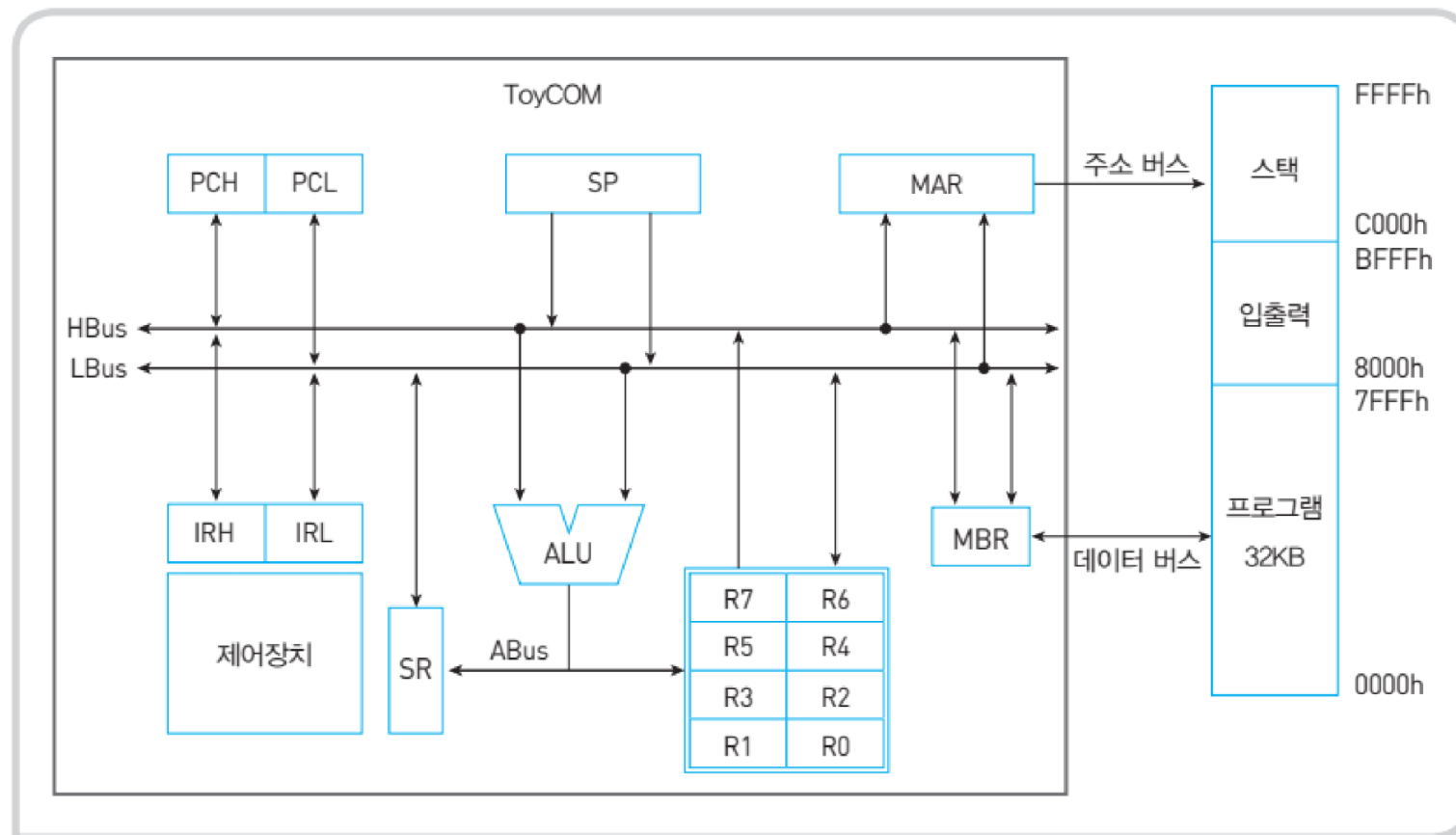


그림 7-1 ToyCOM 프로그래머 모델

7.6.2 제어신호 할당

제어신호 표현 규칙

[I/O][레지스터 이름][H/L/A]

// '/'는 여러 개 중 하나를 선택함을 나타낸다.

- 접두어 [I/O]
 - I: 내부 버스에서 레지스터로 데이터를 적재
 - O: 레지스터의 값을 내부 버스로 출력
- 레지스터 이름
 - 제어신호를 할당할 레지스터 이름
- 접미어 [H/L/A]
 - 사용하는 내부 버스를 특별히 표시할 필요가 있을 때 접미어를 붙인다.
 - H: HBus L: LBus A: 둘 다.

예

- IIRL: LBus에서 IR 레지스터로 데이터로 적재하는 제어신호
- OMBRL: MBR 레지스터의 값을 LBus로 출력하는 제어신호

추가 제어신호

PC

- (IncPC) $PC \leftarrow PC+1$: 명령어를 인출할 때, PC 값을 증가시킨다.
- (LdVec) $PC \leftarrow 2 \times n$: 인터럽트 요청이 있을 때 PC에 인터럽트 서비스 루틴의 시작 주소를 적재한다.
- (AddPC) $PC \leftarrow PC+IRH[10:8]:IRL$: 조건분기 명령어에서 PC 값에 변위를 더한다. 변위는 $IR[10:0]$ 에 저장되어 있다.

SP

- (DesSP) $SP \leftarrow SP-1$: 스택에 푸시할 때 SP의 값을 감소시킨다.
- (IncSP) $SP \leftarrow SP+1$: 스택에서 팝할 때 SP의 값을 증가시킨다.

연산기

- 연산기가 동작하지 않음: NOP
- 단항 마이크로오퍼레이션: INC, DEC, NEG, NOT, SHL, SHR, ASL, ASR
- 이항 마이크로오퍼레이션: ADD, ADC, SUB, SBC, AND, OR, XOR

제어신호 목록

표 7-18 제어신호 목록

동작 대상	제어신호		
	레지스터 입력	레지스터 출력	기타
PC	IPCH, IPCL	OPCH, OPCL	IncPC, AddPC, LdVec
IR	IIRH, IIRL	OIRH, OIRL	
SP		OSP	IncSP, DecSP
SR	ISRL	OSRL	CLC, STC, CLI, STI
Rx	IRxL, IRxA	ORxH, ORxL	INC, DEC, NEG, NOT, SHL, SHR, ASL, ASR NOP, ADD, ADC, SUB, SBC, AND, OR, XOR
MAR	IMAR		
MBR	IMBRH, IMBRL	OMBRH, OMBRL	
기억장치			RdMem, WrMem

제어신호를 부여한 데이터 경로

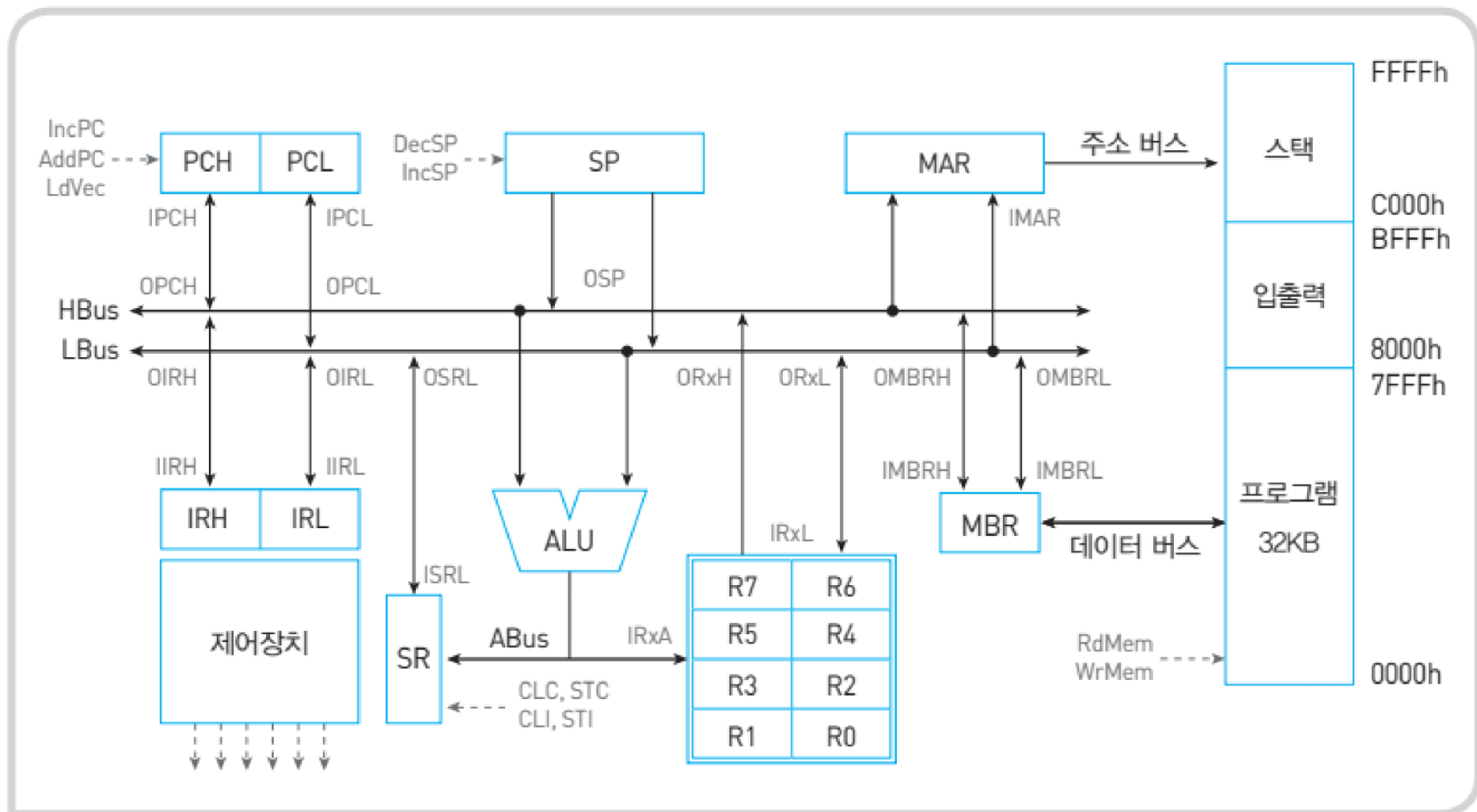


그림 7-10 제어신호를 부여한 데이터 전달 경로

7.6.3 검증

검증 과정

- 예제 프로그램에 대하여
- 명령어 실행 사이클(인출단계, 실행단계)에 따라
- 제어신호가 적절히 생성되어
- 명령어가 올바르게 실행되는지 검사한다.

예제 프로그램

번호	주소:	기계어 코드	명령어	기능 설명
1.	0000:	80FE	BR 100h	// 100h 번지로 무조건 분기
2.	0100:	0920	LDI R1, #20h	// R1 ← 20h
3.	0102:	0A40	LDI R2, #40h	// R2 ← 40h
4.	0104:	F8FA	CALL 200h	// CALL 200h
5.	0106:	0001	HALT	// 프로그램 종료
6.	0200:	2828	ADD R0, R1, R2	// R0 ← R1+R2
7.	0202:	4800	RET	// 리턴

그림 7-11 예제 프로그램

인출단계 동작

표 7-19 인출단계 제어신호

마이크로오퍼레이션	제어신호		
	입력	출력	기타
MAR $\leftarrow$ PC	IMAR	OPCH, OPCL	
MBR $\leftarrow$ Mem(MAR), PC $\leftarrow$ PC + 1			RdMem, IncPC
IRL $\leftarrow$ MBR	IIRL	OMBRL	
MAR $\leftarrow$ PC	IMAR	OPCH, OPCL	
MBR $\leftarrow$ Mem(MAR), PC $\leftarrow$ PC + 1			RdMem, IncPC
IRH $\leftarrow$ MBR	IIRH	OMBRH	

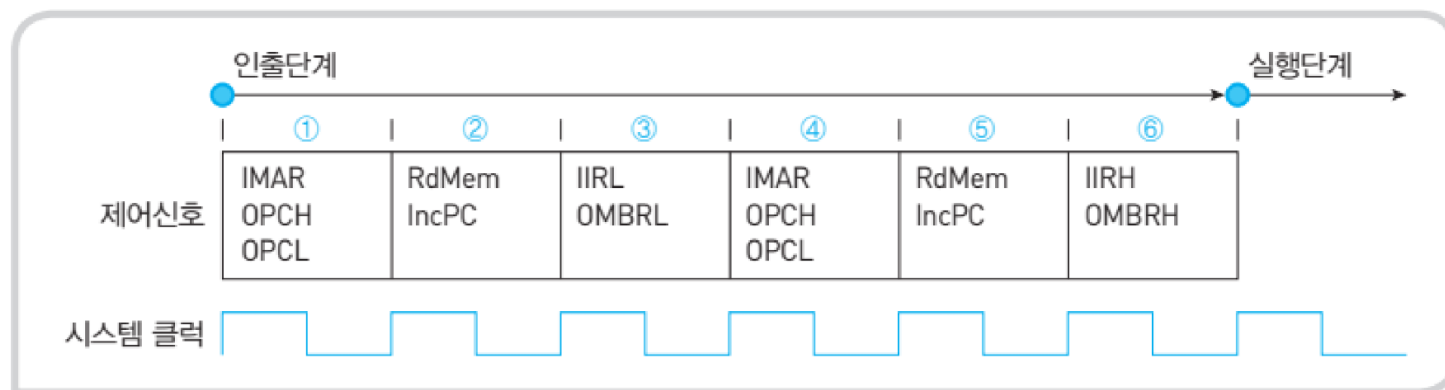


그림 7-12 인출단계의 타이밍 다이어그램

명령어 실행단계의 동작

표 7-20 명령어 실행단계의 제어신호

명령어	단계	마이크로오퍼레이션	제어신호		
			입력	출력	기타
BR 100h	1	$PC \leftarrow PC + \text{offset}$		OIRH, OIRL	AddPC
LDI R1,#20h	1	$R1 \leftarrow 20h$	IR1L	OIRL	
LDI R2,#40h	1	$R2 \leftarrow 40h$	IR2L	OIRL	
CALL 200h	1	$SP \leftarrow SP - 1$			DecSP
	2	$MAR \leftarrow SP$	IMAR	OSP	
	3	$MBR \leftarrow PCH$	IMBRH	OPCH	
	4	$Mem(MAR) \leftarrow MBR$			WrMem
	5	$SP \leftarrow SP - 1$			DecSP
	6	$MAR \leftarrow SP$	IMAR	OSCP	
	7	$MBR \leftarrow PCL$	IMBRL	OPCL	
	8	$Mem(MAR) \leftarrow MBR$			WrMem
	9	$PC \leftarrow PC + \text{offset}$		OIRH, OIRL	AddPC
HALT	1	-	-	-	-
ADD R0,R1,R2	1	$R0 \leftarrow R1 + R2$	IR0A	OR1H, OR2L	ADD
RET	1	$MAR \leftarrow SP$	IMAR	OSP	
	2	$MBR \leftarrow Mem(MAR)$		IMBRL	RdMem
	3	$PCL \leftarrow MBR, SP \leftarrow SP+1$	IPCL	OMBRL	IncSP
	4	$MAR \leftarrow SP$	IMAR	OSP	
	5	$MBR \leftarrow Mem(MAR)$		IMRBL	RdMem
	6	$PCH \leftarrow MBR, SP \leftarrow SP+1$	IPCH	OMBRH	IncSP